

COMP3011 Coursework 1 — Technical Report

Student Name: Han Lin

Student ID: 201690928

Module: COMP3011

Project Title: Travel Planner API

Submission Date: 15/03/2026

Repository:

<https://github.com/coraloose/comp3011-travel-api>

API Documentation (PDF):

https://github.com/coraloose/comp3011-travel-api/blob/main/API_Documentation.pdf

1. Overview

This project implements a RESTful Web API for a travel planning scenario. The system supports the end-to-end workflow of (i) creating and managing points of interest (“Places”), (ii) building travel “Trips” and organising itinerary items per day, (iii) collecting user interactions such as bookmarks and reviews, (iv) recording trip expenses and producing a budget summary, and (v) providing analytics endpoints for ranking places and generating an itinerary automatically.

The primary goal is to demonstrate a coherent API design with persistent storage, meaningful data relationships, appropriate HTTP status codes, and analytical queries beyond basic CRUD operations.

2. Technology Stack and Rationale

Programming Language: Python 3.12

Web Framework: FastAPI + Uvicorn

Persistence Layer: SQLAlchemy ORM

Database: SQLite

Rationale:

- **FastAPI** was chosen due to its strong alignment with RESTful API development, automatic OpenAPI generation, and interactive Swagger UI for debugging and demonstration. It also provides robust request validation through Pydantic.

- **SQLAlchemy** was selected to implement a clear object-relational mapping (ORM) layer, which improves maintainability and allows the project to scale to other relational databases (e.g., PostgreSQL) with minimal changes.
- **SQLite** was used as a low-configuration database suitable for coursework and local testing. It enables rapid iteration without external dependencies.

The API documentation is produced using the automatically generated OpenAPI schema via Swagger UI, and a curated PDF documentation is included for submission requirements.

3. Data Model and Relationships

The database schema is designed around travel planning entities and user interactions. Key tables include:

1. **Place**

Represents a point of interest (POI).

Fields: id, city, name, category.

2. **Trip**

Represents a travel plan with a date range.

Fields: id, name, start_date, end_date.

3. **TripPlace** (association entity)

Links a Trip to a Place while storing itinerary metadata. This design normalises the relationship and supports additional attributes.

Fields: id, trip_id, place_id, day, planned_order, note.

4. **Bookmark**

Represents a user bookmarking a place.

Fields: id, place_id, user_name, created_at.

Constraint: unique (place_id, user_name) to prevent duplicate bookmarks by the same user.

5. **Review**

Represents a user review of a place with a rating.

Fields: id, place_id, user_name, rating, comment, created_at.

6. Expense

Represents a trip-level expense record.

Fields: id, trip_id, category, amount, currency, date, note.

Relationships :

- Trip 1—N TripPlace N—1 Place
- Place 1—N Bookmark
- Place 1—N Review
- Trip 1—N Expense

This schema supports both operational endpoints (CRUD) and analytical endpoints (ranking and aggregations).

4. Key Design and Implementation Decisions

4.1 RESTful Endpoint Design

The API follows conventional REST patterns:

- POST creates resources (returns 201 Created).
- GET retrieves resources (returns 200 OK).
- PUT/PATCH updates resources (returns 200 OK with updated object).
- DELETE removes resources (returns 204 No Content).

Resource identifiers use path parameters (e.g., `/places/{place_id}`), while optional filtering uses query parameters (e.g., `/places?city=Paris&category=museum`).

4.2 Validation and Error Handling

Input validation is implemented using Pydantic schemas. Invalid payloads automatically return

422 Validation Error.

The system implements domain-level errors consistently:

- **404 Not Found** when a referenced resource does not exist (e.g., Trip not found, Place not found).
- **400 Bad Request** for logically invalid input (e.g., `end_date < start_date` in Trip creation/update).
- **409 Conflict** for duplicate bookmarks (same `user_name` and `place_id`), enforced by a database unique constraint and handled by catching `IntegrityError`.

4.3 Analytics Endpoints (Non-CRUD Functionality)

Beyond standard CRUD, several analytics endpoints are provided:

1. Budget Summary

`GET /analytics/trips/{trip_id}/budget-summary?currency=EUR`

Uses SQL aggregation (`SUM`) with `GROUP BY category` to compute both total spend per category and the overall grand total.

2. City Rankings

`GET /analytics/cities/{city}/top-bookmarked`

Computes bookmark counts per place in a city using `COUNT(bookmarks)` and grouping.

`GET /analytics/cities/{city}/top-rated?min_reviews=3`

Computes average rating and review count per place in a city using `AVG(rating)` and `COUNT(reviews)` with a minimum review threshold (`HAVING count >= min_reviews`).

These endpoints demonstrate relational joins and aggregation queries, which are key learning outcomes for database-backed API systems.

4.4 Itinerary Generation

`POST /analytics/trips/{trip_id}/generate-itinerary` automatically assigns itinerary days for `TripPlace` entries where `day` is currently null. A simple round-robin strategy is used based on trip duration in days:

- `num_days = (end_date - start_date) + 1`
- Unassigned items are distributed across days `1..num_days`

This provides a practical “automation” feature while remaining deterministic and easy to explain during oral assessment.

5. Testing Approach

Testing was conducted primarily through Swagger UI. The following were verified: (i) CRUD operations for Places and Trips/TripPlace, (ii) validation and error codes (400/404/409/422), and (iii) correctness of analytics outputs (budget totals and city rankings). Budget results were cross-checked by ensuring the sum of category totals matches the returned grand total.

6. Limitations and Future Improvements

While the system meets the coursework requirements, several improvements are identified:

1. Authentication and authorisation

- The current implementation uses a lightweight `user_name` field to simulate user interactions without implementing authentication.
- A production system would use secure authentication (e.g., JWT) and user IDs.

2 . Database migrations

- Tables are created via SQLAlchemy `create_all()`(prototype approach).
- Future improvement: adopt Alembic for schema migrations and versioning.

3.Pagination and performance

- Listing endpoints return full arrays.
- Future improvement: add pagination (`limit`, `offset`) and indexes tuned to query workload.

4.Monetary precision

- `amount` uses floating-point which may introduce rounding errors.
- Future improvement: use `Decimal/Numeric` for financial accuracy.

5.Deployment

- The current work focuses on local execution.
- Future improvement: deploy on Render/Railway/PythonAnywhere and provide a public base URL.

7. GenAI Use

During development, I used GenAI as a structured design and review assistant rather than a code generator. The workflow followed iterative cycles: requirement decomposition, candidate solution generation, student implementation, and verification. GenAI helped translate the coursework brief into milestones (Place CRUD, Trip/TripPlace, Bookmark/Review, Expense/budget summary, and analytics) and define acceptance criteria (status codes and response schemas) before implementation. It also supported design-space exploration; I adopted an explicit association table (TripPlace) for normalisation and extensibility and enforced bookmark uniqueness at the database level, reflected as a 409 Conflict on duplicates. GenAI was used to assist debugging (imports, type annotations, merge conflicts) and to review REST conventions, while all changes were implemented and validated by the student. Finally, controlled Swagger-based tests were used to verify correctness, including aggregation checks for budget summaries and ranking behaviour for city analytics.